# CS437 Semester Project: Market Predictions using Trend Deterministic Data Preparation and Machine Learning Techniques: An Analysis of Methods

Chance Spreitzer Luke Langan Jack Duensing

12/23/2025

## 1 Introduction

### 1.1 Purpose

The purpose of this project is to deliver a fully implemented and analyzed approach for predicting stock market prices, as exemplified in a given paper: *Predicting stock and stock price index movement using Trend Deterministic Data Preparation and machine learning techniques, Jigar Patel et. al.* The paper uses 10 created features based on stock data. Data used includes the open, close, high, and low prices for 2 important Indian stock indices, as well as 2 important Indian individual stocks. The models used by the paper, and therefore this project, are Support Vector Machine (SVM), Artificial Neural Networks (ANN), Naïve-Bayes, and Random Forests. During our implementation we found a critical discrepancy. The paper implies it is forecasting the next day's price, but the methodology only supports classifying the current day's movement. The features are calculated using the current day's data, which is then used to predict the current days outcome. To replicate the paper's results, our primary task is to implement this descriptive model and validate the paper's accuracy claims. After the validation of the initial paper, we look to a meta-analysis of our paper, combined with multiple others, to provide direction for implementation of our own prediction model.

### 1.2 Objectives

- Replicating the paper's data pipeline and preprocessing
- Implement and achieve an accuracy greater than or equal to 90% with the four models implemented by the paper

SVM

ANN

Naïve-Bayes

Random Forests

- Analyze the methodology of the paper and its possible short sights
- Create our own model and features to improve and build upon the paper

  Achieve an accuracy of +60% as shown in the meta-analysis

# 2 Methodology

## 2.1 Data Acquisition

The data being used for our use case takes an American perspective. The tickers to predict are; Microsoft (MSFT), Amazon (AMZN), S&P 500 (GSPC), and Dow Jones (DJI). These stocks were chosen as they mirror the choices in the paper for the American market. GSPC and DJI are similarly large, market indicating indices, MSFT is a leading competitor in the technology sector, and AMZN provides a premier example of a large market reaching stock. The stock data (close, high, low, open, and volume) is collected from yfinance over the time frame of 10 years from 2015 to 2025. This 10 year swath will provide more than enough data to implement the models.

## 2.2 Leaky Model

### 2.2.1 Feature Engineering

The initial paper uses 10 technical indicators as features in order to predict the same day's stock price. These 10 features are in 2 different formats, and our code contains functions to calculate all 10 of them in both formats

**Continuous Data** is used as the raw data, and are normalized between -1 and 1 in preparation for use as features in the models

**Trend Deterministic Data** is used as discrete data, and is tabulated by converting the values of the continuous data to a hard +1 or -1. This rounding is determined by whether the data indicates up movement or down movement.

The 10 indicators are as follows:

- Simple 10 day moving average (SMA)
- Weighting 10 day moving average (WMA)
- Momentum
- Stochastic K
- Stochastic D
- Relative Strength Index (RSI)
- Moving Average Convergence Divergence (MACD)
- Larry William's R
- Accumulation/Distribution (A/D) Oscillator
- Commodity Channel Index (CCI)

#### 2.2.2 Implementation

**Naïve-Bayes** was implemented using Gaussian Naïve Bayes for the continuous data as done in the paper. The trend deterministic data was implemented via a multivariate Bernoulli distribution. The continuous results are shown in Table 1, and the trend deterministic results are shown in Table 2.

| | Accuracy | F1 |
|---|---|---|
| Microsoft (MSFT) | 0.634383 | 0.683438 |
| Amazon (AMZN) | 0.618241 | 0.656 |
| S&P 500 (GSPC) | 0.625504 | 0.681756 |
| Dow Jones (DJI) | 0.630347 | 0.68544 |

Table 1: Naïve Bayes Continuous Results

**Random Forests** was implemented by tuning n_trees. The top 3 results from 20 to 200 trees is shown in Tables 3 and 4. Table 3 shows continuous results, and Table 4 shows Trend results.

| | Accuracy | F1 |
|---|---|---|
| Microsoft (MSFT) | 0.919714 | 0.924458 |
| Amazon (AMZN) | 0.915739 | 0.920659 |
| S&P 500 (GSPC) | 0.926073 | 0.930649 |
| Dow Jones (DJI) | 0.90779 | 0.914328 |

Table 2: Naïve Bayes Trend Results

| | Trees | Accuracy | F1 |
|---|---|---|---|
| Microsoft (MSFT) | 80 | 0.794913 | 0.807750 |
| | 60 | 0.794118 | 0.807721 |
| | 100 | 0.794118 | 0.807435 |
| Amazon (AMZN) | 120 | 0.810811 | 0.821321 |
| | 100 | 0.810016 | 0.820436 |
| | 140 | 0.810016 | 0.819894 |
| S&P 500 (GSPC) | 180 | 0.819555 | 0.837509 |
| | 200 | 0.819555 | 0.837741 |
| | 140 | 0.815580 | 0.833333 |
| Dow Jones (DJI) | 40 | 0.796502 | 0.816881 |
| | 180 | 0.796502 | 0.815296 |
| | 200 | 0.795707 | 0.814440 |

Table 3: Top 3 Continuous Tree Counts

**SVM** was implemented with both Polynomial and Radial Basis Function (RBF) kernels. The C value in SVM controls the tradeoff between maximizing the margin and minimizing classifications. A high C value, like 100, will have a higher penalty for misclassifying training points which leads to a smaller margin. A large C value will typically result in overfitting. A small C value has a lower penalty for misclassifying training points which results in a wider margin and can underfit. We used C values ranging from 0.1 to 100 for both kernels. Similarly, the gamma value in SVM determines the reach of influence each point has on the decision boundary. High gamma values can overfit while low gamma values will underfit. We used gamma values ranging from 0.1 to 5 for the RBF kernel.

Kernels are slightly different in their implementation. The Polynomial kernel maps data to a higher dimensional space using a polynomial function which makes it suitable for data with polynomial relationships. It is less flexible than RBF but can be less computationally expensive. On the other hand, the RBF kernel is where the data is mapped to an infinite dimensional feature space. It can classify data that is

| | Trees | Accuracy | F1 |
|---|---|---|---|
| Microsoft (MSFT) | 80 | 0.924483 | 0.928945 |
| | 140 | 0.924483 | 0.928945 |
| | 160 | 0.924483 | 0.928945 |
| Amazon (AMZN) | 60 | 0.90461 | 0.911243 |
| | 80 | 0.90461 | 0.911243 |
| | 100 | 0.90461 | 0.911243 |
| S&P 500 (GSPC) | 20 | 0.924483 | 0.929682 |
| | 40 | 0.924483 | 0.929889 |
| | 60 | 0.924483 | 0.929786 |
| Dow Jones (DJI) | 20 | 0.923688 | 0.929099 |
| | 60 | 0.923688 | 0.929099 |
| | 40 | 0.922893 | 0.928519 |

Table 4: Top 3 Trend Tree Counts

not linearly separable. A linear hyperplane will separate the classes, which is done with the gamma value.

Table 5 and 6 show the SVM results, for the continuous data and the trend data respectively. Note: each row is a result, the rows with degree indicate a Polynomial kernel, and the rows with gamma indicate a RBF kernel.

| | C | Degree/Gamma | Accuracy | F1 |
|---|---|---|---|---|
| Microsoft (MSFT) | 10 | Degree: 1 | 0.832123 | 0.847507 |
| | 100 | Gamma: 0.1 | 0.829701 | 0.842184 |
| Amazon (AMZN) | 10 | Degree: 1 | 0.824052 | 0.837313 |
| | 100 | Gamma: 0.1 | 0.822437 | 0.833837 |
| S&P 500 (GSPC) | 100 | Degree: 1 | 0.846651 | 0.860088 |
| | 100 | Gamma: 0.1 | 0.833737 | 0.846726 |
| Dow Jones (DJI) | 10 | Degree: 1 | 0.840194 | 0.856105 |
| | 10 | Gamma: 0.1 | 0.840194 | 0.854839 |

Table 5: Continuous Polynomial/RBF Kernel SVM

**ANN** was implemented with an extensive number of parameters checked. The epochs used were varied across 1000, 2000, and 3000; these epochs simply determine the rounds taken over the data set. The momentum constant is a hyperparameter that allows the model to converge more quickly, as well as assisting

| | C | Degree/Gamma | Accuracy | F1 |
|---|---|---|---|---|
| Microsoft (MSFT) | 5 | Degree: 1 | 0.925278 | 0.928571 |
| | 1 | Gamma: 0.1 | 0.931638 | 0.934551 |
| Amazon (AMZN) | 1 | Degree: 1 | 0.914944 | 0.920799 |
| | 0.1 | Gamma: 0.1 | 0.910970 | 0.961542 |
| S&P 500 (GSPC) | 0.5 | Degree: 1 | 0.918919 | 0.924332 |
| | 1 | Gamma: 0.1 | 0.924483 | 0.930198 |
| Dow Jones (DJI) | 0.5 | Degree: 1 | 0.910970 | 0.916418 |
| | 0.5 | Gamma: 0.5 | 0.922893 | 0.928519 |

Table 6: Trend Polynomial/RBF Kernel SVM

gradient descent in avoiding local minima. The momentum constant is a factor by which the previous weight change is carried over to the current weight change; it was varied across 0.1, 0.3, 0.5, 0.7, and 0.9. The last hyperparameter tested was the hidden neuron count, this was varied across 10, 30, 50. As you can see from tables 7 and 8 there was clearly one set of hyperparameters tested that performed the best. Tables 7 and 8 show results of the tuned model on continuous and trend data respectively

| | Epochs | Neurons | Momentum Constant | Accuracy | F1 Score |
|---|---|---|---|---|---|
| MSFT | 1000 | 30 | 0.5 | 0.8128 | 0.8196 |
| AMZN | 1000 | 30 | 0.5 | 0.8119 | 0.8561 |
| GSPC | 1000 | 30 | 0.5 | 0.8434 | 0.8561 |
| DJI | 1000 | 30 | 0.5 | 0.8232 | 0.8414 |

Table 7: Continuous ANN

| | Epochs | Neurons | Momentum Constant | Accuracy | F1 Score |
|---|---|---|---|---|---|
| MSFT | 1000 | 30 | 0.5 | 0.9269 | 0.9320 |
| AMZN | 1000 | 30 | 0.5 | 0.8975 | 0.9052 |
| GSPC | 1000 | 30 | 0.5 | 0.9213 | 0.9269 |
| DJI | 1000 | 30 | 0.5 | 0.9245 | 0.9269 |

Table 8: Trend ANN

### 2.3 New Model

In this section we look at optimizing the methods of the previous paper to allow for prediction of the next day's market price. Then we look at combining ap-

proaches in a stacked model, in an attempt to further extend our prediction capabilities.

### 2.3.1 Feature Engineering

Using our knowledge of the original paper's features, as well as the critiques of the meta-analysis, we have created 6 new features to use for our own implementation. We also decided to swap our Amazon ticker for the Apple ticker in this section.

- Simple moving average
  - 14 day
  - 50 day
- Weighted moving average: 14 day
- Momentum: 10 day
- Volatility
- Relative Strength Index
- Multi-day Lags
  - 1 day
  - 2 day
  - 3 day
  - 5 day

Choosing features is as important in this situation as the features themselves. For models of this size, working over this much data, and with this many features, we fear two inevitabilities; overfitting and training time. A model that uses too many features makes it difficult for the model to accurately weight each feature correctly. This results in overfitting by the models, as they are attempting to fit too much data, and therefore do not see the underlying trends within it. Therefore we have decided to limit the features given to each model. The second problem is training time; we do not have access to advanced machines to run these models. Therefore we must select the best features to give to each model. This leads us to feature selection.

**Feature Selection** for these models was done on a case by case basis for the 4 models in the original paper; Naive Bayes, SVM, Random Forest, and ANN. For each model we want to find the optimal features that will increase final accuracy. For each model we loop through all features, and select the n best ones. This n depends on the computational cost of the model. For Naive Bayes, as the least computationally heavy, we use 7 features. Next, for SVM and Random Forests, middle of the pack computationally, we use 5 features. Finally, for ANN, the heaviest model, we limit to just 4 features. The next 4 tables show the best results of this feature selection for each model, on each ticker.

### 2.3.2 Implementation

Naive Bayes, shown in table 9, implemented to predict the next day's price. Each ticker shows its used features.

| | Features | Accuracy (%) | Precision | Recall | F1 Score |
|---|---|---|---|---|---|
| AAPL | SMA_50<br>WMA_14<br>Momentum_10<br>Lag_3<br>Lag_5 | 63.97 | 0.6691 | 0.6790 | 0.6740 |
| MSFT | RSI_14<br>Momentum_10<br>Lag_5 | 62.75 | 0.6420 | 0.7750 | 0.7023 |
| GSPC | Momentum_10<br>Volatility_14<br>Lag_1<br>Lag_3<br>Lag_5 | 62.96 | 0.6220 | 0.8467 | 0.7172 |
| DJI | Momentum_10<br>Volatility_14<br>Lag_1<br>Lag_2<br>Lag_3<br>Lag_5 | 61.13 | 0.6176 | 0.7721 | 0.6863 |

Table 9: Naive Bayes, predicting next day's price

For the SVM model, shown in table 10, we also loop through the C hyperparameter.

| | Features | C Value | Accuracy (%) | Precision | Recall | F1 Score |
|---|---|---|---|---|---|---|
| AAPL | Momentum_10<br>Lag_1<br>Lag_2 | 0.5 | 64.78 | 0.6448 | 0.7970 | 0.7129 |
| MSFT | RSI_14<br>Momentum_10<br>Lag_1<br>Lag_2<br>Lag_5 | 0.5 | 63.77 | 0.6431 | 0.8107 | 0.7172 |
| GSPC | Momentum_10<br>Lag_1<br>Lag_2<br>Lag_5 | 5 | 65.18 | 0.6441 | 0.8321 | 0.7261 |
| DJI | Momentum_10<br>Volatility_14<br>Lag_1<br>Lag_2<br>Lag_5 | 1 | 64.57 | 0.6502 | 0.7721 | 0.7059 |

Table 10: SVM, predicting next day's price

For the Random Forest implementation, shown in table 11, we also loop through the tree count from 10 to 200 with a step of 10

For ANN, shown in table 12, we must also loop through epochs, momentum, and hidden layers. Hidden layer possibilities are 10, 50, or 100. Momentum possibilities are 0.3, 0.6, or 0.9. Epoch possibilities are 1000 or 2000. Learning rate was kept constant at 0.1.

This implementation for predicting the next day's stock price is in line with our goals of +60% accuracy. The ANN model performs the best, and the Random Forest Model, the worst. However, we think we can still perform better, using a stacked model. This stacked method will combine 2 models. The base learner is an individually optimized version of a previously implemented model. Then the predictions from that model feed into the meta learner as features. The meta learner aims to weigh the credibility of each model. Meta learners used are logistic regression, random forests, and gradient boosting. Table 13 shows those results.

| | Features | N Trees | Accuracy (%) | Precision | Recall | F1 Score |
|---|---|---|---|---|---|---|
| AAPL | RSI_14<br>Momentum_10<br>Lag_5 | 100 | 59.51 | 0.6212 | 0.6717 | 0.6454 |
| MSFT | RSI_14<br>Volatility_14<br>Lag_2<br>Lag_5 | 50 | 60.12 | 0.6416 | 0.6714 | 0.6562 |
| GSPC | SMA_50<br>Momentum_10<br>Volatility_14 | 150 | 59.92 | 0.5964 | 0.8577 | 0.7036 |
| DJI | RSI_14<br>Momentum_10<br>Lag_5 | 100 | 57.69 | 0.6121 | 0.6324 | 0.6221 |

Table 11: Random Forest, predicting next days' price

We can see that Microsoft (MSFT) was the only ticker to perform better with the stacked model than just the initial single model. This bump of 2.23 % gives MSFT a success in Table 13. The other 3 tickers performed best with just the plain ANN model. Another item of note in Table 13 is the increased recall across the board. This increased recall means that the model will be more successful in predicting up trends, therefore this could result in more often "losing". Losing refers to buying stock that is predicted to go up, but then the stock does not behave as predicted.

# 3 Result Analysis

## 3.1 Leaky Model Analysis

The implementation of the leaky model was misguided. We had initially assumed that the implementation of the 4 models (Naive Bayes, SVM, Random Forests, and ANN) was to predict the next day's stock price. Upon a detailed read of the paper we discovered that the paper aimed to predict the price of the ticker for the same day. This was a large hurdle in the implementation of these models. Once we discovered the models intended purpose the results were much easier to understand. The models were trained on both continuous and trend deterministic data. Across the board the trend deterministic data yielded better results, sometimes stretching into the 90s for accuracy readings. The best results were shared by Random Forests and SVM.

| | Features | Hidden Layers | Epochs | Momen. | Acc. % | Prec. | Recall | F1 Score |
|---|---|---|---|---|---|---|---|---|
| AAPL | Momentum_10<br>Lag_1<br>Lag_2 | 100 | 1000 | 0.6 | 65.79 | 0.6624 | 0.7675 | 0.7111 |
| MSFT | Momentum_10<br>Volatility_14<br>Lag_2<br>Lag_5 | 10 | 1000 | 0.9 | 65.18 | 0.6646 | 0.7786 | 0.7171 |
| GSPC | Momentum_10<br>Lag_1<br>Lag_2<br>Lag_5 | 100 | 1000 | 0.9 | 64.98 | 0.6490 | 0.8029 | 0.7178 |
| DJI | Momentum_10<br>Lag_1<br>Lag_2<br>Lag_5 | 10 | 1000 | 0.3 | 66.40 | 0.6699 | 0.7684 | 0.7158 |

Table 12: ANN, predicting next day's price

The leaky model also has a fatal flaw. One of the features used contained data about that same day's stock price. This represented a considerable form of data leakage. The model had the answers for what it was trying to predict, however in a mathematically altered format. This problem is what allowed such high accuracy values, even on models not known for being accurate in this setting. This was a serious oversight in our opinion, and was the catalyst for the implementation of our own predicting model.

### 3.2 New Model Analysis

The implementation of our new model was directed from the beginning to achieve a next day prediction. Our goal was +60% accuracy predicting the next day's price using the same 4 models (Naive Bayes, SVM, Random Forests, and ANN), and the same 4 tickers. We manufactured a new list of features from what was successful in the previous implementation. However, we could not allow all of these features to be used by every model. We knew this could result in overfitting, and also huge computational cost from some of the heavier models. We decided to limit the maximum number of features any one round of the model could use. Each model got a chance to use all of the features up to the maximum allocation. This allowed the best features to be picked by each model for each ticker. The results of these new tests are displayed in tables 9-12. The best single model accuracy was a value

| | Meta Learner | Accuracy (%) | Precision | Recall | F1 Score | Success or Failure |
|---|---|---|---|---|---|---|
| AAPL | Logistic Regression | 65.59 | 0.6554 | 0.7860 | 0.7148 | Failure |
| MSFT | Random Forest | 67.41 | 0.6854 | 0.7857 | 0.7321 | Success |
| GSPC | Logistic Regression | 64.37 | 0.6494 | 0.7774 | 0.7076 | Failure |
| DJI | Logistic Regression | 65.38 | 0.6593 | 0.7684 | 0.7097 | Failure |

Table 13: Implementing the stacked model

of 66.40% predicting the Dow Jones by the ANN model. The ANN model created the best results, but was also the most computationally heavy. ANN also produced the best single stock prediction, predicting Apple at an accuracy of 65.79%. SVM was the next closest performing model. Random Forests performed the worst for the next day predictions.

We discovered a meta-analysis paper that compared leaky data sets to genuine data sets. The paper, *Stock Market Prediction Using Machine Learning* analyzed multiple approaches to stock market prediction both with and without leaky data sets. Both rounds of our results, the direct implementation of our initial paper and our new model, were consistent with the prediction figures in that meta analysis.

### 3.3 Implementation Challenges

**Trend Data Transformation** Our primary implementation challenge is to translate continuous stock data into discrete outputs. The paper uses the trend deterministic layer used to normalize the data into values of -1 or 1 to show an increase or decrease. For example, SMA and WMA need to be -1 if the price is below the average and +1 if the price is above. Stochastic Oscillators will be +1 if the oscillators value at the time 't' is greater than its value at 't-1'. The RSI feature has multiple parts such as; the trend will be +1 if RSI < 30, therefore oversold, the trend will be -1 if RSI > 70, therefore overbought. The trend is then determined by comparing RSI at 't' to 't-1'.

# 4 Discussion

## 4.1 Difficulties/Hurdles

**Initial Difficulties** When selecting our project, we set out to create a predictive model for stock and indices prediction, as the paper and abstract would seem to imply. The models we set up were initially configured to take in features of today's prices $X(t)$ and predict tomorrow's price movement, $y(t+1)$. This method proved consistently poor results that were not at all consistent with the paper. All models, despite tuning and feature engineering, struggled to perform over an $\approx 55\%$ accuracy. This was a significant hurdle in the early stages of implementation; this accuracy is barely better than a coin flip. Our confusion was compounded when we noticed that the Naïve Bayes model performed the best on the paper, when we know that it is one of the simplest models of the four. At this point our methodology was consistent with the paper and we could not even think of approaching the paper's fabled $90\%$ accuracy. An email was drafted and sent to the author with no reply, but upon further examination of the paper's terminology we found that the models were descriptive, not predictive. The output of the model was the same day's price movement. However, the input features of the model hold a mathematically transformed version of that price. The model had its own answers, albeit in a different format. This represents a significant form of data leakage. This discovery resolved our initial confusion, and we later learned that the $\approx 55\%$ is, in fact, correct and realistic for a true predictive attempt using the paper's features.

# 5 Conclusion

Predicting next day stock prices has been a very interesting problem for decades; people make their entire living off of predicting the fluctuation in the market. With the advent of machine learning becoming increasingly more accessible, fast, and with machines dedicated for the task, using it to predict ticker prices is a logical jump. Many papers exist on this problem, all trying to use years of data to guess what, at times, seems to be random meandering up and down of the market. As we see in the implementation in this project, machine learning allows those seemingly random ups and downs to be predicted with odds better than the coin flip most people think it is.

This project aimed to implement and analyze the findings of *Predicting stock and stock price index movement using Trend Deterministic Data Preparation and machine learning techniques.* Upon analysis of the paper and its methods, we

found two issues. The first was that the paper was not attempting to predict the next day's stock price at all, it as simply trying to predict the same day's price. The second issue was that it used an approach with a considerable data leak. The paper was using features that already contained the information it was trying to predict. This allowed the paper to have very inflated accuracy scores, that did not seem possible. The overcoming of these issues was a considerable challenge in the implementation and analysis of this paper.

After the implementation of the initial paper, we were not satisfied with just predicting the same day's price. We wanted to create a model that predicted the next day's price. We manufactured new features with no data leaks, and created next day prediction models based on the foundation of the initial paper. This produced new results that coincided with our goals of +60% prediction accuracy. We consulted a meta-analysis of these leaky methods vs genuine methods by the title of *Stock Market Prediction Using Machine Learning*. This allowed us to see that the results we were getting with our initial next day implementation of the first paper's methods were not incorrect but much more in line with what current literature supported on predicting next day ticker prices. The creation of our own features and hyper parameter tuning allowed us to even further refine these results.

# 6 References

Patel, J., Shah, S., Thakkar, P., & Kotecha, K. (2015). Predicting stock and stock price index movement using Trend Deterministic Data Preparation and machine learning techniques. Expert Systems with Applications, 42, 259–268. https://doi.org/10.1016/j.eswa.2014.07.040

Subasi, A., Amir, F., Bagedo, K., Shams, A., & Sarirete, A. (2021). Stock Market Prediction Using Machine Learning. Procedia Computer Science, 194, 173–179. https://doi.org/10.1016/j.procs.2021.10.071